

PYTHON FUNDAMENTALS

Interview Study Guide

Concept-only full-day curriculum

Goal: build the language model in your head before you start coding practice.

How to use this guide

Study each session in order. After each session, close the notes and explain the concepts aloud from memory. Do not move on merely because the material looks familiar; move on when you can explain it precisely and answer the checkpoint questions without notes.

1. One-Day Concept Study Plan

This is intentionally intensive because you said you have most of the day available. The plan is concept-first: no coding drills are required today. The purpose is to build a coherent mental model of Python so that later programming practice reinforces understanding instead of becoming syntax memorization.

Time	Block	Concept focus
08:30-09:15	Session 1	How Python executes code; interpreter, bytecode, runtime, dynamic typing
09:15-10:15	Session 2	Variables, object references, identity, type system, mutability
10:15-11:00	Session 3	Numbers, booleans, None, operators, truthiness
11:00-12:00	Session 4	Strings: immutability, indexing/slicing, methods, formatting, Unicode
12:00-13:15	Session 5	Core collections: list, tuple, set, dict and when to use each
13:15-14:00	Break	Lunch / mental reset
14:00-14:45	Session 6	Control flow: if/elif/else, loops, break/continue/pass, match
14:45-16:00	Session 7	Functions: parameters, returns, scope, LEGB, *args/**kwargs, defaults
16:00-16:45	Session 8	Equality, identity, copying, aliasing, shallow vs deep copy
16:45-18:00	Session 9	OOP fundamentals: classes, objects, methods, inheritance, composition
18:00-18:45	Session 10	Exceptions and defensive error handling
18:45-19:30	Break	Dinner / short walk
19:30-20:15	Session 11	Modules, packages, imports, pip and virtual environments
20:15-21:00	Session 12	Iterables, iterators, generators and comprehensions

21:00-21:40	Session 13	Files, context managers and resource management
21:40-22:20	Session 14	Decorators and first-class functions - conceptual level
22:20-23:00	Session 15	Memory management, garbage collection and the GIL
23:00-23:30	Session 16	Closed-notes recall + interview checkpoints

If the day becomes too long

Do not cut Sessions 1-9. Those form the actual Python foundation. Sessions 10-15 can spill into the next morning if needed. Retention is more important than touching every heading once.

What not to study today

- Django / Flask / FastAPI frameworks
- Advanced async networking
- Metaclasses and descriptors in depth
- Heavy competitive-programming problems
- NumPy/Pandas internals
- Machine-learning libraries
- Packaging/publishing to PyPI
- CPython C-extension internals

These topics are useful later, but they are poor substitutes for being able to reason about Python objects, functions, collections, scope, exceptions, and classes.

2. Session 1 - What Python Actually Is

Before syntax, understand the execution model. Interview answers become much stronger when you can distinguish the Python language from a particular implementation such as CPython.

Language vs implementation

Python is a programming language specification and ecosystem. CPython is the most widely used implementation of Python and is written primarily in C. Other implementations exist, such as PyPy. In everyday interviews, “Python” usually means CPython unless the distinction matters.

From source code to execution

1. You write source code in a .py file or provide code interactively.
2. The Python parser converts source text into an internal syntax representation and validates syntax.
3. CPython compiles the code into Python bytecode - instructions for the Python virtual machine, not native CPU instructions.
4. The CPython evaluation loop executes those bytecode instructions.
5. Imported modules may have cached bytecode files under `__pycache__` to avoid recompiling unchanged source unnecessarily.

Interview precision

Calling Python simply “interpreted” is acceptable at a high level, but incomplete. CPython normally compiles source to bytecode and then executes that bytecode using its virtual machine.

Dynamic typing

Python is dynamically typed: type information is associated with objects at runtime, and a variable name can be rebound to objects of different types. This is different from saying Python has no types - Python has a strong runtime type system.

Strong typing

Python generally does not silently combine unrelated types just because conversion might be imaginable. For example, a string and an integer are distinct types; operations must be valid for the objects involved or an exception is raised.

Indentation is syntax

Python uses indentation to define blocks. Indentation is therefore semantic, not merely formatting. PEP 8 conventionally recommends four spaces per indentation level.

Interview checkpoint

- Is Python compiled or interpreted? Give the precise CPython answer.
- What does “dynamically typed” mean?
- Does dynamic typing mean Python is weakly typed?
- What is bytecode and where does it sit between source code and execution?
- Why is indentation more significant in Python than in C-like languages?

3. Session 2 - Variables, Objects, References and Types

This is one of the highest-value sessions in the entire guide. A surprising number of “tricky Python questions” reduce to a single idea: names refer to objects.

Variables are names bound to objects

A useful mental model is not “a variable is a box containing a value.” In Python, an assignment binds a name to an object. Multiple names can refer to the same object, and a name can later be rebound to another object.

Core mental model

Think: NAME -> OBJECT. Assignment changes bindings. Whether an operation affects other names depends on whether the referenced object is mutable and whether both names point to that same object.

Every object has three important properties

Property	Meaning	Typical way to inspect
Identity	Which specific object it is during its lifetime	<code>id(obj)</code> ; identity comparison with <code>is</code>
Type	What operations/behavior the object supports	<code>type(obj)</code> , <code>isinstance(obj, T)</code>
Value	The object's logical data/content	Compared using <code>==</code> when equality is defined

Type annotations

Python supports type hints such as `x: int`, but ordinary CPython does not enforce these annotations at runtime by default. They primarily help readers, IDEs, linters, and static type checkers such as `mypy` or `pyright`. The runtime object still determines behavior.

Mutable vs immutable

Usually immutable	Usually mutable
<code>int</code> , <code>float</code> , <code>bool</code>	<code>list</code>
<code>str</code> , <code>tuple</code>	<code>dict</code>
<code>frozenset</code> , <code>bytes</code>	<code>set</code>
<code>NoneType</code>	most user-defined objects unless designed otherwise

“Immutable” means the object itself cannot be modified after creation. A name can still be rebound to a different immutable object. Also, a tuple is immutable as a container, but it may contain references to mutable objects.

Object identity vs value

Two separate objects can have equal values. Identity asks whether they are literally the same object; equality asks whether their values are considered equivalent under the type’s equality semantics.

Interview checkpoint

- What exactly happens conceptually when Python executes an assignment?
- Can two variables point to the same list? What are the consequences?
- What is the difference between `type(obj)` and `isinstance(obj, T)`?
- Are Python type hints enforced automatically at runtime?
- Can a tuple contain a list, and if so is the tuple still immutable?

4. Session 3 - Numbers, Booleans, None and Operators

Core numeric types

Type	Meaning	Important detail
int	Arbitrary-precision integer	Not limited to fixed 32-bit/64-bit range in normal Python usage
float	Double-precision floating-point number	Uses binary floating-point; many decimal fractions are approximate
complex	Complex number	Has real and imaginary components
bool	Boolean value: True or False	bool is a subclass of int in Python
Decimal	Decimal arithmetic from decimal module	Useful when exact decimal behavior matters, e.g. finance
Fraction	Rational number from fractions module	Represents numerator/denominator exactly

Arithmetic operators

Know the semantic distinction between / (true division), // (floor division), % (modulo), ** (exponentiation), and unary +/-. Floor division rounds toward negative infinity, which matters for negative operands.

Comparison operators

Python supports ==, !=, <, <=, >, >= and chained comparisons. Chained comparisons are evaluated conceptually as a conjunction while evaluating the middle expression only once.

Logical operators and truthiness

and, or, and not operate using truth-value semantics. Importantly, and/or return one of their operands rather than forcing the result to bool. This enables patterns such as selecting a fallback value, but can surprise beginners.

Falsy values commonly include False, None, numeric zero, and empty collections/strings. Most other objects are truthy unless their class customizes truth testing.

None

None is a singleton object representing the absence of a value. Compare against it with “is None” or “is not None”, not value equality as a style rule.

Floating-point warning

Do not assume $0.1 + 0.2$ is represented exactly as 0.3 in binary floating-point. For tolerant comparisons, use an appropriate strategy such as `math.isclose` when the domain permits it.

Interview checkpoint

- Why can floating-point equality be dangerous?
- What is the difference between `/` and `//`?
- What values are falsy in Python?
- Why is `"x is None"` preferred to `"x == None"`?
- What do `and`/or actually return?

5. Session 4 - Strings

Strings appear in almost every Python interview because they combine immutability, sequence behavior, Unicode, formatting, slicing, and common algorithmic operations.

Strings are immutable Unicode sequences

A str represents a sequence of Unicode code points. Operations that appear to modify a string actually create a new string object. This is why repeated concatenation in a large loop can be inefficient compared with accumulating pieces and joining them.

Indexing and slicing

Indexing returns one-character strings. Positive indexes count from the start; negative indexes count backward from the end. Slicing uses the conceptual form start:stop:step, where stop is excluded.

Common conceptual method families

Family	Examples	What to understand
Case	lower, upper, casefold	Return new strings; casefold is stronger for caseless Unicode comparisons
Whitespace	strip, lstrip, rstrip	Remove matching boundary characters, not arbitrary internal whitespace
Search	find, index, count	find returns -1 when absent; index raises ValueError
Split/join	split, rsplit, join	join belongs to the separator string and combines an iterable of strings
Replace	replace	Returns a new string because str is immutable
Tests	startswith, endswith, isdigit, isalpha	Return booleans for common validation questions

Formatting

Know f-strings conceptually as the modern default for readable interpolation. Also recognize str.format and the older percent-formatting style. F-strings can evaluate expressions and support format specifications.

str vs bytes

str represents text; bytes represents raw byte values. Encoding converts text to bytes. Decoding converts bytes back to text using a character encoding such as UTF-8. This distinction matters for files, network protocols, and binary data.

Interview checkpoint

- Why are strings immutable?
- What does a negative index mean?
- Why is the stop index excluded in a slice?
- What is the difference between str and bytes?
- When would join be preferable to repeated concatenation?

6. Session 5 - Core Collections

Your goal is not to memorize methods. Your goal is to choose the right collection based on ordering, mutability, uniqueness, access pattern, and hashability.

List

A list is a mutable ordered sequence. It is appropriate when order matters and elements may be added, removed, replaced, or reordered. Index lookup is direct; membership testing is generally linear because values must be searched.

Tuple

A tuple is an immutable ordered sequence. It is useful for fixed-size records, multiple return values, unpacking, and cases where immutability communicates intent. A tuple can be hashable only if all of its elements are hashable.

Set

A set is a mutable collection of unique hashable elements. It is designed for fast membership tests and mathematical set operations such as union, intersection, difference, and symmetric difference. Sets do not provide list-style positional indexing.

Dictionary

A dict maps unique hashable keys to values. Modern Python dictionaries preserve insertion order as part of the language guarantee. Dicts are optimized for key-based lookup and are central to Python programming.

Collection	Ordered?	Mutable?	Duplicates?	Primary strength
list	Yes	Yes	Yes	General ordered mutable sequence
tuple	Yes	No	Yes	Fixed/immutable sequence and record-like grouping
set	No positional order	Yes	No	Fast membership and uniqueness
dict	Insertion order	Yes	Keys unique	Fast mapping from keys to values

Hashability

Hashable objects have a hash value that remains stable during their lifetime and implement compatible equality behavior. Dictionary keys and set elements must be hashable. This is why a list cannot normally be a dict key: lists are mutable and therefore unhashable.

Complexity intuition

Operation	Typical complexity	Reason
list index access	$O(1)$	Direct position lookup
list membership	$O(n)$	Potential scan through elements
append to list	Amortized $O(1)$	Occasional resize cost spread across appends
dict/set lookup	Average $O(1)$	Hash-table based
dict/set worst case	Can degrade	Depends on hashing/collisions and implementation behavior

Interview trap

Do not say “sets are unordered” if what you mean is “sets have no supported positional indexing / sequence order contract.” The key practical point is that you should not treat a set as a sequence whose element position is meaningful.

Interview checkpoint

- List vs tuple - when would you choose each?
- Why can a tuple sometimes be a dictionary key while a list cannot?
- Why are set membership tests typically faster than list membership tests?
- Do dictionaries preserve insertion order?
- What does hashable mean?

7. Session 6 - Control Flow

Conditional branching

if / elif / else chooses a path based on truth-value testing. Python does not require the condition to be literally True or False; any object can participate through truthiness.

Loops

for iterates over an iterable. This is fundamentally different from the traditional C-style “counter loop” mental model. while repeats while a condition remains truthy.

Loop control

Keyword	Purpose
break	Exit the nearest enclosing loop immediately
continue	Skip the rest of the current iteration and continue with the next
pass	Do nothing; syntactic placeholder where a statement is required
else on loop	Runs if the loop terminates normally, not via break

match / case

Structural pattern matching, introduced in modern Python, can match values and data shapes. It is not simply a switch statement clone; patterns can destructure data and bind names. For a fundamentals interview, know what it is and when it improves readability.

Short-circuit evaluation

and and or stop evaluating as soon as the overall result can be determined. This matters both for efficiency and for safety when the right-hand expression assumes something established by the left-hand expression.

Interview checkpoint

- How is a Python for loop different conceptually from a C-style for loop?
- Difference between break, continue, and pass?
- What does a loop else block mean?
- What is short-circuit evaluation?
- Is match/case exactly the same as switch/case?

8. Session 7 - Functions

Functions are one of the most frequently probed Python areas because they expose scope, object passing, defaults, closures, and Python's first-class-function model.

Functions are objects

A function defined with `def` creates a function object and binds a name to it. Because functions are first-class objects, they can be assigned to variables, stored in collections, passed as arguments, and returned from other functions.

Parameters vs arguments

Parameters are names in the function definition. Arguments are the actual values/objects supplied during a call. Keep this distinction precise in interviews.

Parameter categories

Category	Meaning
Positional-only	Must be passed by position; marked before / in a signature
Positional-or-keyword	May be passed either way; the most common kind
Variadic positional: <code>*args</code>	Collects extra positional arguments into a tuple
Keyword-only	Must be supplied by name; appear after <code>*</code> or <code>*args</code>
Variadic keyword: <code>**kwargs</code>	Collects extra keyword arguments into a dict

Return values

A function returns `None` implicitly if execution reaches the end without an explicit return. A “multiple-value return” is conceptually a tuple being returned, often unpacked immediately by the caller.

Scope and LEGB

When resolving an unqualified name, Python conceptually searches Local, Enclosing, Global, then Built-in scopes. The `global` keyword allows rebinding a module-level name; `nonlocal` allows rebinding a name in an enclosing function scope.

Argument passing model

A productive description is “call by object reference” or “call by sharing.” Function parameters become local names bound to the same objects supplied by the caller. Rebinding the parameter does not rebind the caller's name; mutating a shared mutable object can be visible to the caller.

Default arguments

Default argument expressions are evaluated once when the function definition executes, not every time the function is called. A mutable default object can therefore persist across calls. The common defensive pattern is to use `None` as a sentinel and create the mutable object inside the function.

Lambda

`lambda` creates a small anonymous function limited to a single expression. Use it where a short callable improves readability; do not force complex logic into lambdas.

Critical interview point

Python is not usefully described as pass-by-value or pass-by-reference in the C++ sense. The function receives references to objects. Rebinding a local parameter and mutating the referenced object are different operations.

Interview checkpoint

- Explain `*args` and `**kwargs`.
- What is LEGB?
- Why are mutable default arguments dangerous?
- What happens if a function has no return statement?
- How would you explain Python argument passing accurately?
- What does it mean that functions are first-class objects?

9. Session 8 - Equality, Identity, Aliasing and Copying

== vs is

== asks whether two objects are equal according to their equality semantics. is asks whether two references point to the exact same object. Use is primarily for singleton identity checks such as None.

Aliasing

Aliasing occurs when multiple names reference the same object. With mutable objects, a mutation through one alias is observable through the others.

Shallow copy

A shallow copy creates a new outer container but reuses references to the nested objects. Therefore changes to the outer container structure are independent, while mutations to shared nested mutable objects may still appear through both copies.

Deep copy

A deep copy recursively copies contained objects as appropriate, attempting to produce an independent object graph. It is more expensive, and not every object can or should be meaningfully deep-copied.

Operation	Outer object new?	Nested mutable objects shared?
Simple assignment	No	Yes - same object graph
Shallow copy	Yes	Usually yes
Deep copy	Yes	Generally copied recursively

Interview checkpoint

- Difference between == and is?
- What is aliasing?
- Shallow copy vs deep copy?
- Why might a shallow copy still appear to “change unexpectedly”?
- Why should identity not usually be used to compare ordinary numeric/string values?

10. Session 9 - Object-Oriented Programming Fundamentals

For interviews, prioritize the object model and method semantics over memorizing design-pattern vocabulary.

Class and object

A class defines behavior and provides the mechanism used to create instances. An instance is an object whose type is that class. Instance attributes generally store per-object state; class attributes live on the class and are shared through attribute lookup unless shadowed by an instance attribute.

self

self is the conventional name for the instance parameter of an instance method. It is not a reserved keyword, but the convention is universal and should be followed. Calling an instance method through an instance causes that instance to be supplied as the first argument through method binding.

__init__ vs __new__

__new__ is responsible for creating/returning a new instance; __init__ initializes an already-created instance. Most application code customizes __init__, not __new__.

Method types

Method	First automatic argument	Typical role
Instance method	self	Operate on instance state
Class method	cls	Operate on class-level state; alternative constructors
Static method	None	Utility logically associated with the class but not requiring instance/class state

Inheritance and MRO

Inheritance lets a class derive behavior from base classes. Python supports multiple inheritance. Method Resolution Order (MRO) defines the order in which classes are searched for attributes/methods and makes cooperative super() calls predictable.

Polymorphism and duck typing

Python commonly relies on behavioral compatibility: if an object provides the operations a piece of code expects, the exact concrete type may not matter. This is the essence of duck typing. Formal inheritance is useful but not always necessary for polymorphic code.

Encapsulation

Python relies heavily on conventions rather than strict access modifiers. A leading underscore indicates an implementation detail. A double-leading-underscore name inside a class triggers name mangling, mainly to reduce accidental collisions in subclasses rather than to create true privacy.

Composition vs inheritance

Composition models “has-a” relationships by containing/collaborating with other objects. Inheritance models an “is-a” relationship. Composition is often easier to evolve because it reduces coupling to a class hierarchy.

Dunder methods

Special methods such as `__str__`, `__repr__`, `__len__`, `__iter__`, `__eq__`, and `__hash__` integrate user-defined classes with Python protocols. You normally trigger them indirectly through built-ins/operators rather than calling them explicitly.

Interview checkpoint

- Class attribute vs instance attribute?
- What does `self` actually represent?
- `__new__` vs `__init__`?
- Instance vs class vs static method?
- What is duck typing?
- What is MRO and why does it matter?
- Composition vs inheritance?
- `__str__` vs `__repr__`?

11. Session 10 - Exceptions and Error Handling

Exceptions are objects

Runtime errors are represented by exception objects arranged in an inheritance hierarchy. Handling should normally target the narrowest exception types you can recover from meaningfully.

try / except / else / finally

Block	When it runs
try	Code that may raise an exception
except	Runs when a matching exception propagates from the try block
else	Runs if the try block finishes without an exception
finally	Runs whether an exception occurred or not, subject to process-level termination edge cases

raise

raise explicitly raises an exception. A bare raise inside an exception handler re-raises the currently handled exception. Exception chaining can preserve causal context when translating one exception into another.

EAFP

Python culture often favors EAFP - “Easier to Ask Forgiveness than Permission” - when attempting the operation and handling a predictable exception is clearer and race-safe. This should not be abused to hide unexpected errors.

Bad pattern

Avoid a bare except: unless you have a very specific top-level containment reason. Catching everything can hide programming errors, interrupts, and system-exiting exceptions. Prefer specific exception classes and a deliberate recovery strategy.

Interview checkpoint

- Difference between except, else, and finally?
- Why is catching Exception broadly often undesirable?
- What does raise without an argument do inside except?
- What is EAFP?
- When should you create a custom exception type?

12. Session 11 - Modules, Packages, Imports and Environments

Module

A module is typically a Python file that provides its own global namespace. Importing a module executes its top-level code the first time it is loaded in a process and creates/uses a module object cached in `sys.modules`.

Package

A package organizes related modules under a namespace. Modern Python supports regular packages and namespace packages. For fundamentals, understand the directory/module relationship and relative vs absolute imports.

Import styles

Recognize import module, from module import name, and aliased imports. Prefer explicit imports that keep dependencies readable. Wildcard imports make provenance and static analysis harder and are generally discouraged outside carefully designed interactive/use cases.

`__name__` and main guard

A module has a `__name__`. When a file is executed as the main program, `__name__` is set to `"__main__"`. When imported, `__name__` is its module name. The main guard lets code run only when the file is executed directly, not when imported.

pip and virtual environments

pip installs Python distributions/packages from package indexes or other sources. A virtual environment creates an isolated Python environment so project dependencies do not have to collide with system/global packages or other projects.

Interview checkpoint

- What is the difference between a module and a package?
- What happens the first time a module is imported?
- Why use `if __name__ == "__main__":`?
- Why use a virtual environment?
- Why are wildcard imports usually discouraged?

13. Session 12 - Iterables, Iterators, Generators and Comprehensions

Iterable

An iterable is an object capable of producing an iterator, typically through `__iter__`, or otherwise supporting Python's iteration protocol. Lists, tuples, strings, dicts, sets, and many custom objects are iterable.

Iterator

An iterator represents a traversal state. It produces the next item via `__next__` and raises `StopIteration` when exhausted. An iterator is generally single-pass: once consumed, continuing from it does not restart automatically.

Generator

A generator is a convenient way to produce iterator behavior. A generator function contains `yield`; calling it returns a generator object without executing the full function immediately. Iteration resumes execution around each `yield`, preserving local state between steps.

Lazy evaluation

Generators compute values on demand rather than materializing an entire result collection up front. This can reduce memory usage and enable pipelines over very large or potentially unbounded data streams.

Comprehensions

List, set, and dict comprehensions provide concise declarative construction from iterables, optionally with filtering. Generator expressions use parentheses and produce values lazily instead of creating a whole list.

Helpful iteration tools

Tool	Purpose
<code>enumerate</code>	Pairs items with an index/counter
<code>zip</code>	Combines elements from multiple iterables positionally
<code>range</code>	Represents an arithmetic sequence lazily and efficiently
<code>iter</code>	Obtains an iterator from an iterable
<code>next</code>	Requests the next item from an iterator

Interview checkpoint

- Iterable vs iterator?
- Why is an iterator usually single-pass?
- What does `yield` do?

- Generator vs list - key tradeoff?
- List comprehension vs generator expression?
- Why use enumerate instead of manually managing a counter?

14. Session 13 - Files and Context Managers

File objects

Opening a file returns a file object that manages an underlying operating-system resource. File modes describe reading/writing/appending, text vs binary access, and whether the file is created or truncated.

Text encoding

Text files require encoding bytes to/from str. UTF-8 is a common modern choice, but robust software should be explicit when the environment or data source does not guarantee the default encoding.

Context managers

A context manager defines setup/teardown behavior around a block, used through the with statement. It is ideal for resources that must be released even when an exception occurs.

Conceptually, the context-management protocol is implemented through `__enter__` and `__exit__`. File objects support it, so leaving the with block closes the file reliably.

Why “with open(...)” matters

It is not just stylistic sugar. It expresses deterministic resource cleanup and remains correct if control leaves the block because of an exception.

Interview checkpoint

- Why use with when working with files?
- Text mode vs binary mode?
- What role does encoding play?
- What are `__enter__` and `__exit__`?
- Why is relying only on garbage collection for closing critical resources a poor design?

15. Session 14 - Decorators and Closures

Decorators are often taught as “advanced,” but the conceptual foundation is straightforward once you understand first-class functions and closures.

Closure

A closure is a function that retains access to variables from an enclosing lexical scope even after the outer function has finished. This is enabled by Python’s nested-scope model.

Decorator

A decorator is a callable that takes another callable (or class) and returns a replacement or modified callable/class. The `@decorator` syntax is syntactic sugar for rebinding the function name to the decorator’s result.

Why decorators exist

They are useful for cross-cutting behavior such as logging, authorization, caching, timing, retries, validation, registration, and framework metadata without manually repeating wrapper code at every call site.

Metadata preservation

When writing function decorators, `functools.wraps` is commonly used so the wrapper preserves useful metadata such as the wrapped function’s name, docstring, and introspection relationship.

Interview checkpoint

- What is a closure?
- What does `@decorator` syntax mean conceptually?
- Why are functions being first-class objects necessary for decorators?
- Give practical uses of decorators.
- What problem does `functools.wraps` solve?

16. Session 15 - Memory Management and the GIL

You do not need CPython-implementation depth for a basic interview, but you should be able to explain the headline behavior accurately.

Reference counting

CPython primarily tracks how many references point to an object. When an object's reference count reaches zero, CPython can usually reclaim it immediately.

Cyclic garbage collector

Reference counting alone cannot reclaim cycles in which objects reference one another but are otherwise unreachable. CPython therefore also has a cyclic garbage collector to detect certain unreachable reference cycles.

Object allocation

Python manages memory for objects through its runtime and allocator layers. Application code normally relies on automatic memory management rather than explicit free/delete operations.

The Global Interpreter Lock (GIL)

In the standard CPython build historically used in production, the GIL allows only one thread at a time to execute Python bytecode within a process. This limits CPU-bound parallelism using ordinary Python threads, but does not mean threads are useless. Threads can still overlap waiting on I/O because the GIL may be released during blocking operations and some native extension work.

Threads vs processes vs asyncio - conceptual

Model	Best mental model	Typical fit
threading	Multiple OS threads sharing one process memory	I/O-bound work, blocking libraries, concurrency
multiprocessing	Separate processes with separate interpreter/memory	CPU-bound parallelism when process overhead is acceptable
asyncio	Cooperative event loop with coroutines	High-concurrency I/O with async-compatible operations

Keep the GIL answer scoped

Do not say “Python cannot do multithreading.” Python absolutely supports threads. The important limitation is parallel execution of Python bytecode in a traditional single CPython process for CPU-bound work.

Interview checkpoint

- How does CPython primarily manage object lifetime?

- Why is a cyclic garbage collector needed in addition to reference counting?
- What does the GIL actually restrict?
- Are Python threads useful?
- When would multiprocessing be more suitable than threading?

17. Consolidated Interview Traps

Weak / inaccurate statement	Interview-grade correction
“Python variables store values.”	Better: names are bound to objects; multiple names may reference the same object.
“Python is only interpreted.”	Better: CPython compiles source to bytecode, then executes it in the Python virtual machine.
“is is a faster ==.”	Wrong concept. is checks identity; == checks equality.
“Tuple means everything inside it is immutable.”	The tuple structure is immutable, but referenced elements may themselves be mutable.
“Default parameter values are created on every call.”	They are evaluated when the function definition executes.
“A set is a sorted list without duplicates.”	A set is a hash-based collection of unique hashable elements with no sequence indexing contract.
“Python passes objects by reference.”	Better: call by sharing/object reference; local rebinding and object mutation are different.
“The GIL means Python threads never run concurrently.”	Threads can overlap I/O; the classic GIL constrains simultaneous Python bytecode execution in one process.
“finally only runs after an exception.”	finally runs after try/except flow regardless of whether an exception was raised, barring process-level edge cases.
“Deep copy is always safer.”	Deep copy is costly and can be semantically wrong; copy only when ownership/independence actually requires it.

18. End-of-Day Closed-Notes Recall Test

Do this without searching or looking back. Speak your answers aloud. If you cannot explain a question in roughly 30-90 seconds, mark that concept for review tomorrow morning.

6. What happens from the moment CPython receives a .py source file until bytecode executes?
7. What is the difference between a name, an object, an object's identity, its type, and its value?
8. Dynamic typing vs strong typing?
9. Mutable vs immutable? Give at least four examples of each.
10. Why can floating-point arithmetic produce surprising equality results?
11. Explain truthiness and short-circuit evaluation.
12. Why are strings immutable? str vs bytes?
13. Compare list, tuple, set, and dict by order, mutability, duplicates, and lookup behavior.
14. What does hashable mean? Why can't a list be a dict key?
15. Explain how Python for loops relate to the iteration protocol.
16. What do break, continue, pass, and loop-else mean?
17. Parameters vs arguments? Explain each parameter category.
18. Explain LEGB, global, and nonlocal.
19. Explain call-by-sharing / object-reference semantics.
20. Why are mutable default arguments a problem?
21. Explain == vs is, aliasing, shallow copy, and deep copy.
22. Class vs instance? Class attribute vs instance attribute?
23. __new__ vs __init__?
24. Instance method vs class method vs static method?
25. Duck typing, inheritance, composition, and MRO?
26. Try/except/else/finally semantics?
27. Module vs package? Why use the main guard and virtual environments?
28. Iterable vs iterator vs generator?
29. Why are generators memory-efficient? What is lazy evaluation?
30. What does a context manager guarantee conceptually?
31. What is a closure? What is a decorator?
32. Reference counting vs cyclic garbage collection?
33. What does the CPython GIL restrict, and when are threads still appropriate?

19. Mastery Checklist

Use this checklist before moving to programming exercises. You do not need encyclopedic recall; you need a correct mental model and the ability to explain the following without contradictory statements.

Runtime & object model

- ☐ Source -> bytecode -> runtime execution
- ☐ Names bind to objects
- ☐ Identity vs equality
- ☐ Dynamic + strong typing
- ☐ Mutability and aliasing

Built-in data

- ☐ Numeric types and floating-point caveat
- ☐ Truthiness and None
- ☐ Strings and Unicode
- ☐ List / tuple / set / dict
- ☐ Hashability and complexity intuition

Language flow

- ☐ if / elif / else
- ☐ for and while
- ☐ break / continue / pass / loop-else
- ☐ Short-circuiting
- ☐ match/case purpose

Functions

- ☐ Parameters vs arguments
- ☐ All major parameter kinds
- ☐ Return behavior
- ☐ LEGB
- ☐ Call by sharing
- ☐ Mutable defaults
- ☐ First-class functions and lambda

OOP

- ☐ Class/instance
- ☐ Attributes
- ☐ self
- ☐ __new__ / __init__
- ☐ Method types
- ☐ Inheritance/MRO
- ☐ Duck typing
- ☐ Composition
- ☐ Core dunder purpose

Robust programs

- ☐ Exception flow
- ☐ Specific exception handling
- ☐ Modules/packages/imports
- ☐ Virtual environments
- ☐ Files/context managers

Pythonic iteration

- ☐ Iterable/iterator
- ☐ StopIteration
- ☐ Generators/yield
- ☐ Comprehensions
- ☐ enumerate/zip/range

Runtime concepts

- ☐ Closures/decorators
- ☐ Reference counting
- ☐ Cycle collector
- ☐ GIL
- ☐ Threading vs multiprocessing vs asyncio

20. What Comes Next

Once these concepts are stable, the next phase should be implementation rather than more theory. Start with small prediction questions and short programs that force you to apply object identity, mutability, collection choice, function semantics, and exception flow. After that, move into interview-oriented coding patterns and only then into NumPy/Pandas.

Recommended next sequence

34. Output-prediction questions: mutability, scope, default arguments, slicing, comprehensions, aliasing.
35. Small syntax drills: strings, collections, loops, functions, exceptions.
36. OOP mini-designs: class state, inheritance/composition, dunder methods.
37. Iterator/generator exercises and context-manager usage.
38. Interview coding patterns: frequency maps, two pointers, sets, stacks/queues, sorting, recursion basics.
39. Then begin NumPy and Pandas with a strong distinction between native Python semantics and array/dataframe semantics.

Target standard

For a client interview, “I know Python” should mean you can explain why the language behaves the way it does, predict common edge cases, choose appropriate core data structures, and write readable code without fighting the object model. Syntax alone is not enough.

Appendix A - 60-Second Definitions

Term	Interview-ready definition
Object	A runtime entity with identity, type, and value.
Variable/name	A name bound to an object.
Mutable	Object can change its state/value in place.
Immutable	Object cannot be changed in place after creation.
Hashable	Has a stable hash and compatible equality semantics, allowing use as dict key/set element.
Iterable	Can produce an iterator.
Iterator	Stateful producer of successive items using the iteration protocol.
Generator	A convenient iterator produced by generator functions/expressions; computes lazily.
Closure	Function retaining access to lexical variables from an enclosing scope.
Decorator	Callable that receives and returns/replaces another callable/class.
Module	A unit of Python code with its own namespace, commonly a .py file.
Package	A namespace organizing related modules/subpackages.
Exception	An object representing an abnormal control-flow condition/error.
Context manager	Object/protocol for deterministic setup and teardown around a with block.
GIL	CPython interpreter lock that traditionally permits one thread at a time to execute Python bytecode in a process.
MRO	Method Resolution Order: class-search order used for attribute/method lookup in inheritance hierarchies.

Appendix B - Study Method for Tomorrow

Tomorrow, convert concepts into behavior. For each concept, use this loop: predict -> run -> explain -> modify. Predict the output before executing code. Then explain why the runtime behaved that way. Finally change one condition and predict again. This builds the reasoning interviewers care about more efficiently than copying long scripts.